Université Mohammed V de Rabat
École Nationale Supérieure d'Arts et Métiers

FILIÈRE

INDIA&SD

Ingénierie Numérique en Data Science, Intelligence Artificielle & Santé Digitale

RAPPORT DE MINI-PROJET

Industrie 4.0

Jumeau numérique d'une ligne de production

Simuler le comportement d'une ligne, la superviser en temps réel et tester virtuellement des scénarios d'optimisation

Projet n° 3 de la liste « Projets dans le cadre de la thématique Industrie 4.0 »

Réalisé par

Aymane Issami
Marouan Ezbakhe
Hiba Benzahra
Zaid Hsain
Houssam Kichchou
Mohamed Hamdouch

Encadré par

Prof. Maria Zemzami (ENSAM)

3ème année, S5 — Année universitaire 2026/2027

Remerciements

Au terme de ce mini-projet, nous tenons à remercier toutes les personnes qui ont contribué à sa réalisation.

Nos premiers remerciements s'adressent à **Prof. Maria Zemzami**, qui a proposé et encadré ce projet dans le cadre de la thématique Industrie 4.0. Son cahier des charges — contexte et objectifs, spécifications fonctionnelles et techniques, aspects organisationnels et humains, livrables, planning et critères d'évaluation — a structuré notre démarche du début à la fin, et ce rapport en reprend volontairement le plan.

Nous remercions également le corps professoral de l'École Nationale Supérieure d'Arts et Métiers (École Nationale Supérieure d'Arts et Métiers (ENSAM)) de Rabat, et particulièrement les enseignants de la filière *INDIA&SD*, pour la qualité de la formation dispensée : les compétences en programmation, en modélisation et en analyse de données acquises au cours de notre cursus ont été mobilisées à chaque étape du projet.

Enfin, chaque membre de l'équipe remercie les cinq autres pour la rigueur et l'entraide dont chacun a fait preuve : un jumeau numérique n'existe que si la simulation, la communication, la Trois dimensions (3D) et le tableau de bord parlent le même langage, et cet accord s'est construit ensemble.

A. Issami, M. Ezbakhe, H. Benzahra,
Z. Hsain, H. Kichchou, M. Hamdouch
2026

Résumé

Ce rapport présente la conception et la réalisation d'un **jumeau numérique d'une ligne de production simplifiée**, dans le cadre du mini-projet Industrie 4.0 de l'ENSAM Rabat. La ligne étudiée comporte quatre postes successifs — alimentation (5 s), traitement (8 s), contrôle qualité (15 s) et emballage (7 s) — reliés par un convoyeur, avec des files d'attente de capacité limitée, des pannes aléatoires et un taux de rejet de 5 % au contrôle.

Le système repose sur une architecture en trois briques. Un **modèle de simulation** écrit en Python reproduit le comportement de la ligne par pas de temps de 0,05 s : états des machines (en marche, en attente, bloquée, en panne), blocages de l'amont, propagation des pannes et calcul des Key Performance Indicator (KPI). Un **backend** FastAPI diffuse l'état complet de la ligne dix fois par seconde par WebSocket (WS), reçoit les commandes de l'opérateur et enregistre l'historique de production dans une base SQLite. Une **interface** web affiche la ligne en 3D avec une scène Unity exportée en WebGL et un tableau de bord de supervision synchronisé sur le même flux de données.

Le scénario d'optimisation porte sur le goulot d'étranglement du contrôle qualité. Sur une heure simulée avec pannes et une graine aléatoire identique pour tous les scénarios, la ligne initiale produit **218 produits**. Un contrôle automatisé à 8 s ou un second poste de contrôle en parallèle portent la production à **393** et **394 produits** (+80 %), divisent l'attente au contrôle par 9 à 13 et déplacent le goulot vers le poste de traitement. Ces résultats sont mesurés avec la scène Unity connectée au backend. Un convoyeur adaptatif ne change pas le débit mais réduit l'en-cours moyen de 51 %. La cohérence du modèle est vérifiée par 23 tests automatisés (débit de régime égal au débit théorique à 5 % près, conservation des produits, capacité des files, effet des pannes). Les limites — données entièrement simulées, absence de connexion à des équipements réels — sont discutées en conclusion.

Mots-clés : Industrie 4.0, jumeau numérique, simulation de ligne de production, goulot d'étranglement, supervision temps réel, WebSocket, Unity, WebGL, indicateurs de performance.

Abstract

This report presents the design and implementation of a **digital twin of a simplified production line**, carried out as part of the Industry 4.0 mini-project at ENSAM Rabat. The line has four successive stations — feeding (5 s), processing (8 s), quality control (15 s) and packaging (7 s) — linked by a conveyor, with limited-capacity buffers, random breakdowns and a 5% reject rate at quality control.

The system is built from three components. A **simulation model** written in Python reproduces the behaviour of the line in 0.05 s time steps : machine states (running, idle, blocked, down), upstream blocking, failure propagation and KPI computation. A FastAPI **backend** streams the full line state ten times per second over WS, receives operator commands and stores the production history in an SQLite database. A web **interface** renders the line in 3D with a Unity scene exported to WebGL, together with a supervision dashboard synchronised on the same data stream.

The optimisation scenario targets the quality-control bottleneck. Over one simulated hour with breakdowns and the same random seed for every scenario, the initial line produces **218 products**. An automated 8 s inspection or a second parallel inspection station raise output to **393** and **394 products** (+80%), cut the waiting time at inspection by a factor of 9 to 13, and shift the bottleneck to the processing station. These results were measured with the Unity scene connected to the backend. An adaptive conveyor leaves throughput unchanged but reduces average work-in-progress by 51%. Model consistency is checked by 23 automated tests (steady-state throughput within 5% of theory, product conservation, buffer capacity, failure effects). The limitations — fully simulated data, no connection to real equipment — are discussed in the conclusion.

Keywords : Industry 4.0, digital twin, production line simulation, bottleneck, real-time supervision, WebSocket, Unity, WebGL, performance indicators.

Table des matières

Remerciements i

Résumé ii

Abstract iii

Liste des acronymes viii

Introduction générale 1

1 Contexte et objectifs du projet 2
- 1.1 L'Industrie 4.0 en quelques mots . . . 2
- 1.2 Le jumeau numérique . . . 2
- 1.3 Périmètre du projet . . . 3
- 1.4 La ligne étudiée . . . 3
- 1.5 Objectifs . . . 3

2 Spécifications fonctionnelles et techniques 5
- 2.1 Spécifications fonctionnelles . . . 5
- 2.2 Spécifications techniques . . . 6
  - 2.2.1 Technologies retenues . . . 6
  - 2.2.2 Normes et standards . . . 6
  - 2.2.3 Sécurité et cybersécurité . . . 7
  - 2.2.4 Performances attendues . . . 7

3 Conception du jumeau numérique 8
- 3.1 Architecture générale . . . 8
- 3.2 Modèle de simulation de la ligne . . . 8
  - 3.2.1 Flux physique . . . 8
  - 3.2.2 États d'une machine . . . 9
  - 3.2.3 Pas de temps et pannes . . . 9
  - 3.2.4 Indicateurs calculés . . . 9
- 3.3 Protocole d'échange . . . 10
- 3.4 Répartition des rôles . . . 11

4 Implémentation 12
- 4.1 Organisation du dépôt . . . 12
- 4.2 Le modèle de simulation (`sim.py`) . . . 12
- 4.3 Le backend (`server.py`, `storage.py`) . . . 13
- 4.4 La scène 3D (Unity) . . . 13
  - 4.4.1 Fluidité . . . 14
  - 4.4.2 Difficultés rencontrées . . . 15
- 4.5 Le tableau de bord (`app.js`) . . . 15
- 4.6 Validation du modèle . . . 16

5 Scénario d'optimisation **18**
5.1 Problème identifié . . . 18
5.2 Solutions testées . . . 18
5.3 Protocole de comparaison . . . 18
5.4 Résultats . . . 19
5.5 Analyse . . . 20
5.6 Démonstration dans le jumeau numérique . . . 21

6 Aspects organisationnels et humains **22**
6.1 Formation des opérateurs . . . 22
6.2 Adaptation des processus de travail . . . 22
6.3 Gestion du changement . . . 23
6.4 Organisation de l'équipe projet . . . 23

7 Conclusion et perspectives **24**
7.1 Bilan des livrables . . . 24
7.2 Limites assumées . . . 24
7.3 Perspectives . . . 25
7.4 Compétences mobilisées . . . 25

Bibliographie et webographie **26**

A Planning et ressources **27**
A.1 Planning par phases . . . 27
A.2 Ressources . . . 27
A.3 Critères d'évaluation . . . 28

B Résultats des tests automatisés **29**

C Protocole d'échange **30**
C.1 Commandes WebSocket (navigateur → backend) . . . 30
C.2 Messages backend → clients (page web et scène Unity) . . . 30
C.3 Points d'accès REST . . . 30

Table des figures

3.1 Architecture du jumeau numérique. 8
3.2 Flux des produits dans la ligne modélisée. 8
3.3 Machine à états d'un poste (le voyant de la scène 3D reprend ces quatre couleurs). 9

4.1 Vue d'ensemble de la scène Unity (build WebGL, ligne initiale). Les produits s'accumulent devant le contrôle qualité, le traitement et l'alimentation sont bloqués en amont ; le bac rouge reçoit les rebuts, la palette les produits finis. 14
4.2 États visuels dans la scène Unity. 15
4.3 Les trois onglets du tableau de bord. 16

5.1 Production horaire par scénario, avec et sans pannes. 19
5.2 Le tableau de comparaison dans l'interface (bouton « Lancer la comparaison ») ; la colonne « Réglages actuels » reprend la configuration en cours. . 20

Liste des tableaux

1.1 Les trois niveaux d'intégration selon Kritzinger et al. 2
1.2 Périmètre du projet. 3
1.3 Paramètres initiaux de la ligne. 3

2.1 Couverture des fonctionnalités du cahier des charges. 5
2.2 Choix technologiques. 6
2.3 Exigences de performance et valeurs obtenues. 7

3.1 Paramètres de fiabilité par défaut (`DEFAULT_CONFIG` dans `sim.py`). 9
3.2 Répartition des responsabilités et correspondance dans le code. 11

4.1 Scripts C# de la scène Unity. 14
4.2 Propriétés vérifiées par les tests automatisés. 17

5.1 Comparaison avant / après sur 1 h simulée, pannes activées (graine 7), mesurée avec la scène Unity. 19
5.2 Produits finis sur 1 h avec pannes : scène Unity et simulation hors ligne. . 21

6.1 Parcours de formation proposé. 22

7.1 Bilan des livrables du cahier des charges. 24

A.1 Phases du projet et responsables (planning prévisionnel en semaines). . . . 27
A.2 Ressources nécessaires. 27
A.3 Critères du cahier des charges et éléments de preuve. 28

B.1 Comparaison sur 1 h simulée, pannes désactivées (scène Unity). 29

C.1 Commandes acceptées sur `/ws`. 30
C.2 Application Programming Interface (API) Representational State Transfer (REST) de l'historique. 30

Liste des acronymes

3D	Trois dimensions
API	Application Programming Interface
CSV	Comma-Separated Values
ENSAM	École Nationale Supérieure d'Arts et Métiers
ERP	Enterprise Resource Planning
IHM	Interface Homme-Machine
INDIA&SD	Ingénierie Numérique en Data Science, Intelligence Artificielle & Santé Digitale
IoT	Internet of Things
JSON	JavaScript Object Notation
KPI	Key Performance Indicator
MES	Manufacturing Execution System
MTBF	Mean Time Between Failures
MTTR	Mean Time To Repair
OPC UA	Open Platform Communications Unified Architecture
PLC	Programmable Logic Controller
REST	Representational State Transfer
SQL	Structured Query Language
TRS	Taux de Rendement Synthétique
WIP	Work In Progress (en-cours)
WS	WebSocket

Introduction générale

Modifier une ligne de production réelle coûte cher : chaque essai immobilise des machines, consomme de la matière et fait courir un risque sur la production. L'une des promesses de l'Industrie 4.0 est de déplacer une partie de ces essais vers un modèle virtuel fidèle de l'installation, le **jumeau numérique**, alimenté par des données et capable de répondre à la question « que se passerait-il si ? » avant toute intervention physique.

Ce mini-projet, troisième sujet de la liste proposée par Prof. Maria Zemzami [1], a pour objectif de **modéliser une ligne de production simplifiée sous forme de jumeau numérique, permettant de simuler son comportement et de tester des scénarios d'optimisation**. Trois livrables sont attendus : un modèle de simulation du système physique, une interface synchronisée avec des données simulées, et un scénario d'optimisation testé et documenté.

Nous avons réalisé ces trois livrables sous la forme d'une application complète, développée à six et versionnée dans un dépôt Git : une simulation Python de la ligne, un backend temps réel qui la diffuse par WebSocket et en conserve l'historique, une scène 3D réalisée avec Unity et exportée en WebGL, un tableau de bord de supervision, ainsi qu'un module de comparaison de scénarios et une suite de tests automatisés qui valident la cohérence du modèle. Les résultats d'optimisation présentés dans ce rapport ont été mesurés avec la scène Unity elle-même, connectée au backend.

Le rapport suit le plan du cahier des charges. Le chapitre 1 situe le projet dans l'Industrie 4.0 et en fixe le périmètre et les objectifs. Le chapitre 2 détaille les spécifications fonctionnelles et techniques. Le chapitre 3 présente la conception (architecture, modèle de simulation, protocole d'échange, répartition des rôles), et le chapitre 4 sa réalisation. Le chapitre 5 documente le scénario d'optimisation et ses résultats chiffrés. Le chapitre 6 traite des aspects organisationnels et humains, et le chapitre 7 dresse le bilan, les limites assumées et les perspectives.

Chapitre 1

Contexte et objectifs du projet

1.1 L'Industrie 4.0 en quelques mots

Le terme *Industrie 4.0* désigne la quatrième révolution industrielle, après la mécanisation (vapeur), la production de masse (électricité) et l'automatisation (électronique et informatique). Formalisée en Allemagne au début des années 2010 [2], elle repose sur l'interconnexion des équipements de production, des systèmes d'information et des personnes : les machines communiquent leur état, les données sont collectées et analysées en continu, et les décisions de pilotage s'appuient sur ces données plutôt que sur l'observation ponctuelle.

Parmi les technologies associées — capteurs et objets connectés (Internet of Things (IoT)), informatique en nuage, intelligence artificielle, réalité augmentée, robotique collaborative — le **jumeau numérique** occupe une place centrale [3], parce qu'il relie toutes les autres : il est la représentation numérique vivante de l'installation, sur laquelle on peut observer, analyser et expérimenter.

1.2 Le jumeau numérique

Le concept a été formulé par Michael Grieves au début des années 2000 dans le contexte de la gestion du cycle de vie des produits [4] : un objet physique, son double virtuel, et le flux de données qui les relie. La littérature distingue trois niveaux d'intégration selon la nature de ce flux [5] :

Table 1.1. Les trois niveaux d'intégration selon Kritzinger et al.

Niveau	**Physique → numérique**	**Numérique → physique**
Modèle numérique	Manuel	Manuel
Ombre numérique (*digital shadow*)	Automatique	Manuel
Jumeau numérique (*digital twin*)	Automatique	Automatique

Dans un contexte académique sans équipement physique, le système physique est lui-même remplacé par une simulation. Notre projet se situe donc à l'interface entre ces niveaux : la « ligne » est un modèle de simulation qui joue le rôle du système réel et produit des données en continu, et le jumeau (scène 3D et tableau de bord) se synchronise automatiquement sur ces données et peut renvoyer des commandes (changement de réglage, panne provoquée, réparation) qui modifient le comportement de la ligne. C'est précisément ce que demande le livrable « interface synchronisée avec des données simulées ». La norme ISO 23247 [6] décrit un cadre de référence pour ces jumeaux numériques

de fabrication, dont notre architecture reprend la séparation entre entité observée, collecte des données et applications utilisatrices.

1.3 Périmètre du projet

Table 1.2. Périmètre du projet.

Inclus	Exclu
Ligne de 4 postes en série, convoyeur, files d'attente de capacité limitée	Connexion à des automates (PLC) ou capteurs réels
Pannes aléatoires et provoquées, rejets au contrôle qualité	Maintenance prédictive par apprentissage automatique
Visualisation 3D animée et tableau de bord synchronisés	Gestion des stocks, de la planification et des commandes (ERP, MES)
Historique de production et export des données	Déploiement en production, authentification des utilisateurs
Scénarios d'optimisation comparés dans des conditions identiques	Optimisation économique (coûts, investissements)

1.4 La ligne étudiée

La ligne comprend quatre postes successifs : **alimentation**, **traitement**, **contrôle qualité** et **emballage**. Les produits circulent sur un convoyeur et passent par chaque poste. Les temps de traitement de départ sont ceux de l'énoncé du scénario :

Table 1.3. Paramètres initiaux de la ligne.

Poste	Temps de traitement	Cadence maximale isolée
Alimentation	5 s	12 produits/min
Traitement	8 s	7,5 produits/min
Contrôle qualité	15 s	**4 produits/min**
Emballage	7 s	8,6 produits/min

Dans une ligne en série, le débit est limité par le poste le plus lent : c'est le **goulot d'étranglement**, au sens de la théorie des contraintes [7]. Ici, le contrôle qualité (15 s) borne la ligne à 4 produits/min, soit 240 produits/h, alors que les autres postes pourraient aller plus vite. Les produits s'accumulent devant lui et les postes amont finissent par se bloquer.

1.5 Objectifs

1. **Modéliser** la ligne de façon suffisamment fidèle pour reproduire les phénomènes qui comptent pour la décision : goulot, accumulation, blocage de l'amont, effet des pannes et des rejets.

2. **Superviser** la ligne en temps réel dans une interface 3D et un tableau de bord qui affichent les mêmes données au même instant.
3. **Optimiser** : identifier le goulot, tester plusieurs solutions et mesurer leur effet sur le débit, l'attente et l'en-cours, dans des conditions comparables.
4. **Valider** le modèle par des tests automatisés plutôt que par la seule observation visuelle.

Chapitre 2

Spécifications fonctionnelles et techniques

2.1 Spécifications fonctionnelles

Le cahier des charges propose une liste de fonctionnalités possibles. Le tableau 2.1 indique comment chacune est couverte par le projet.

Table 2.1. Couverture des fonctionnalités du cahier des charges.

Fonctionnalité	Réalisation dans le projet	Statut
Automatisation des processus	Enchaînement automatique des postes, transfert par convoyeur, blocage et reprise sans intervention	Réalisé
Collecte et analyse de données en temps réel	État complet diffusé 10 fois/s, KPI calculés en continu, historique SQLite par minute simulée	Réalisé
Interconnexion des équipements et systèmes	Simulation, backend, scène 3D et tableau de bord reliés par un protocole WS unique	Réalisé
Interface homme-machine intuitive	Scène 3D navigable, voyants d'état, tableau de bord, onglet d'aide, manuel utilisateur	Réalisé
Maintenance prédictive	Pannes simulées, MTBF et MTTR observés par poste ; pas de modèle prédictif	Partiel

Les fonctions détaillées attendues par l'utilisateur sont les suivantes :

— **F1 — Visualiser** la ligne en 3D : machines, convoyeur, produits en mouvement, état de chaque machine (en marche, en attente, bloquée, en panne), avancement du produit en cours.

— **F2 — Superviser** : cadence réelle et théorique, produits finis, rejets, en-cours, goulot, état et statistiques de chaque poste.

— **F3 — Piloter** : pause, vitesse de simulation ($\times 1$ à $\times 20$), remise à zéro, modification des temps de cycle, nombre de machines en parallèle, convoyeur adaptatif, activation des pannes.

— **F4 — Simuler des incidents** : provoquer ou réparer une panne sur un poste choisi.

— **F5 — Comparer** : simuler une heure de chaque scénario dans des conditions identiques et afficher les écarts par rapport à la situation initiale.

— **F6 — Historiser** : conserver chaque exécution, consulter ses courbes et événements, exporter les données en Comma-Separated Values (CSV).

2.2 Spécifications techniques

2.2.1 Technologies retenues

Table 2.2. Choix technologiques.

Brique	Technologie	Justification
Simulation	Python 3.12	Langage maîtrisé par toute l'équipe, lisible pour un modèle métier, sans dépendance
Backend	FastAPI + Uvicorn [8]	Serveur asynchrone léger qui gère à la fois WS et REST
Communication	WebSocket [9]	Canal bidirectionnel persistant et à faible latence : l'état descend, les commandes remontent
Stockage	SQLite	Base SQL embarquée dans la bibliothèque standard, un seul fichier, aucun serveur
3D	Unity 6 (URP), export WebGL [10]	Moteur de jeu standard de l'industrie pour les jumeaux numériques, rendu dans le navigateur sans installation côté poste opérateur
Tableau de bord	HTML, CSS, JavaScript	Même page que la scène Unity, relié au même backend ; le tableau de bord et la 3D reçoivent le même état
Validation	pytest	Tests automatisés reproductibles du modèle et de l'API

Pourquoi Unity avec export WebGL ? Le plan de travail prévoyait Unity, moteur de référence pour les jumeaux numériques. Une première version de la scène avait été écrite avec Three.js ; nous l'avons reconstruite dans Unity pour respecter la consigne et obtenir un rendu plus réaliste (éclairage, matériaux, brouillard, ombres). Pour ne pas séparer la 3D du tableau de bord, la scène est exportée en **WebGL** et affichée dans la même page web que le tableau de bord : le navigateur sert d'unique poste de supervision, sans installation. Un petit pont (`LineBridge`) relie la page et la scène (sélection d'une machine dans les deux sens). La contrepartie est un build plus lourd (environ 12 Mo) et un premier chargement de quelques secondes.

2.2.2 Normes et standards

Le projet s'appuie sur des standards ouverts : le protocole WebSocket (RFC 6455 [9]) pour la communication, JavaScript Object Notation (JSON) pour le format des messages, HTTP/REST pour l'historique, CSV pour l'export. Dans une version industrielle, la collecte des données machines passerait par Open Platform Communications Unified

Architecture (OPC UA) (IEC 62541 [11]), standard de fait de l'interopérabilité industrielle ; l'architecture actuelle isole la source de données dans un seul module (`sim.py`) précisément pour permettre ce remplacement.

2.2.3 Sécurité et cybersécurité

Le serveur écoute uniquement sur l'interface locale (`127.0.0.1`) : il n'est pas exposé au réseau. Les commandes reçues sont validées et bornées côté serveur (numéro de poste, temps de cycle entre 1 et 60 s, 1 à 3 machines, vitesse entre $\times 0{,}25$ et $\times 20$) ; une commande invalide renvoie un message d'erreur sans modifier l'état. Aucune donnée personnelle n'est traitée. Une mise en réseau réelle demanderait en plus une authentification des opérateurs, un chiffrement (`wss://`) et des droits différenciés entre consultation et pilotage ; ces points sont listés dans les perspectives.

2.2.4 Performances attendues

Table 2.3. Exigences de performance et valeurs obtenues.

Exigence	**Cible**	**Obtenu**
Fréquence de mise à jour de l'interface	≥ 5 Hz	10 Hz
Pas de simulation	$\leq 0{,}1$ s	0,05 s
Taille d'un message d'état	< 10 ko	≈ 3 ko
Comparaison de 5 scénarios sur 1 h simulée	< 5 s	$\approx 0{,}3$ s
Écart débit simulé / débit théorique en régime	≤ 5 %	vérifié par test
Écart entre la mesure avec la scène Unity et la simulation hors ligne	≤ 2 %	1,3 % au maximum
Rendu 3D	fluide (cible 30 images/s ou plus)	à mesurer sur le poste de démonstration

Chapitre 3

Conception du jumeau numérique

3.1 Architecture générale

Le système est découpé en trois couches, représentées en figure 3.1 : la **ligne simulée**, qui joue le rôle du système physique ; le **backend**, qui la fait avancer, la diffuse et en garde la mémoire ; et l'**interface**, qui la représente et la pilote.

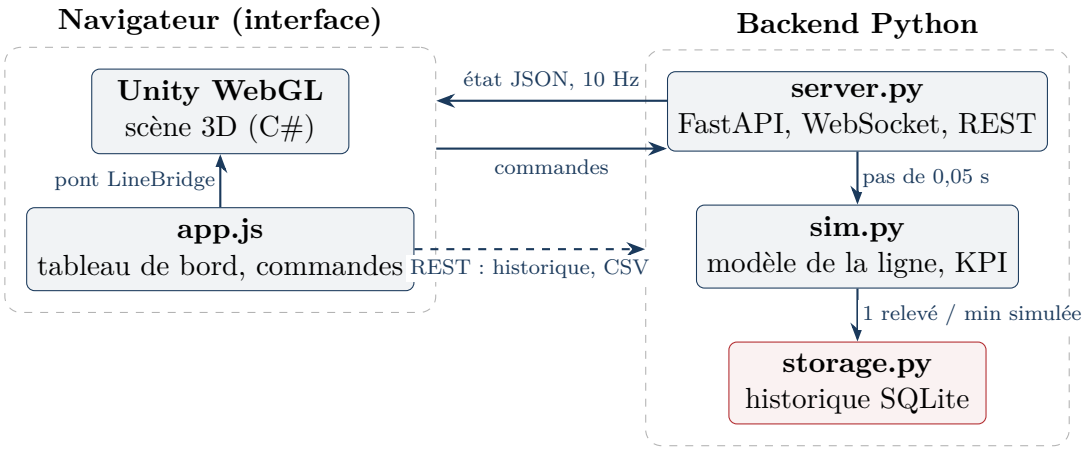


Figure 3.1. Architecture du jumeau numérique.

Le choix structurant est que **la scène 3D et le tableau de bord reçoivent le même état**. Le backend n'envoie pas une information à la 3D et une autre au tableau de bord : il diffuse un état unique de la ligne (machines, files, produits, KPI) sur le même canal WS, et chaque vue en extrait ce qu'elle affiche. La scène Unity se connecte à `/ws` de la même origine que la page (en développement dans l'éditeur, à `ws://127.0.0.1:8000/ws`). Il est donc impossible que la 3D montre une machine en panne pendant que le tableau de bord la déclare en marche.

3.2 Modèle de simulation de la ligne

3.2.1 Flux physique

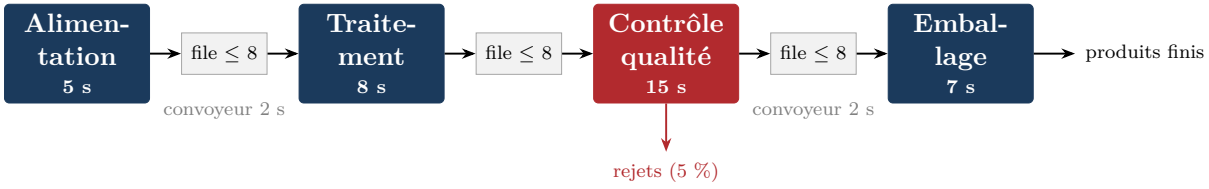


Figure 3.2. Flux des produits dans la ligne modélisée.

L'alimentation dispose d'une matière première illimitée et produit dès qu'elle est libre. Chaque produit terminé est déposé sur le convoyeur (2 s de transfert) puis rejoint la file du poste suivant. Les files ont une **capacité de 8 produits** : si la file aval, convoyeur compris, est pleine, le poste qui vient de terminer garde son produit et se **bloque**. C'est ce mécanisme qui propage un goulot ou une panne vers l'amont, comme sur une vraie ligne. Au contrôle qualité, chaque produit est rejeté avec une probabilité de 5 % et retiré de la ligne.

3.2.2 États d'une machine

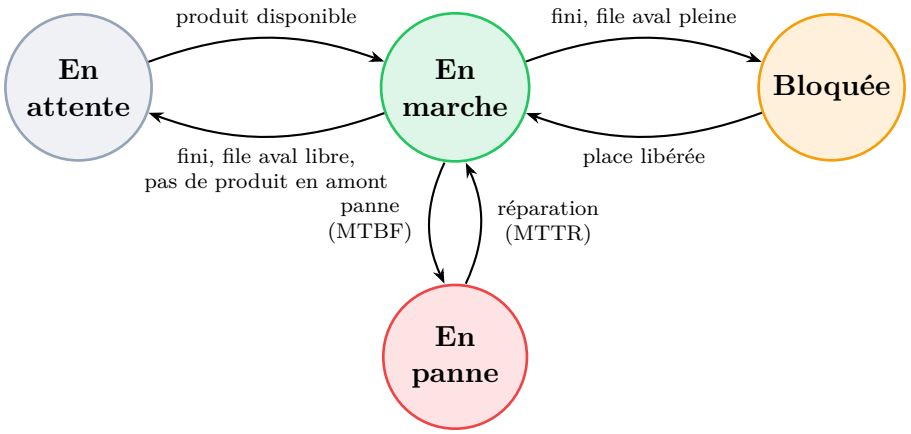


Figure 3.3. Machine à états d'un poste (le voyant de la scène 3D reprend ces quatre couleurs).

3.2.3 Pas de temps et pannes

La simulation avance par **pas de temps fixe** de 0,05 s simulée. À chaque pas, elle traite dans l'ordre les pannes et réparations, les transferts sur le convoyeur, puis les postes **de l'aval vers l'amont** : l'emballage libère sa place avant que le contrôle ne tente d'y déposer un produit, ce qui évite un blocage artificiel d'un pas. Un pas fixe a été préféré à une simulation à événements discrets pure parce qu'il produit naturellement l'avancement continu dont l'animation 3D a besoin (position des produits sur le convoyeur, barre de progression).

Les pannes suivent un modèle classique de fiabilité : le temps de bon fonctionnement avant panne suit une loi exponentielle de moyenne MTBF, et la durée de réparation est tirée uniformément entre 60 % et 140 % du MTTR nominal.

Table 3.1. Paramètres de fiabilité par défaut (`DEFAULT_CONFIG` dans `sim.py`).

	Alimentation	Traitement	Contrôle	Emballage
MTBF (s)	600	500	400	600
MTTR (s)	20	30	25	20

3.2.4 Indicateurs calculés

— **Cadence réelle** : produits finis sur la dernière minute simulée glissante.

— **Cadence théorique** : $60/\max_i(t_i/n_i)$, où t_i est le temps de cycle du poste i et n_i son nombre de machines en parallèle.

— **Taux d'utilisation** d'un poste : temps passé à traiter / (temps écoulé × nombre de machines).

— **Disponibilité** : 1− temps en panne / temps écoulé ; **MTBF et MTTR observés** par poste.

— **En-cours** (Work In Progress (en-cours) (WIP)), **temps de séjour** moyen d'un produit fini, **attente moyenne** dans chaque file.

— **Goulot observé** : le poste qui maximise « utilisation − temps bloqué ». Un poste amont bloqué paraît occupé (il garde son produit) mais il ne limite pas la ligne ; retrancher le temps bloqué évite cette erreur. Pendant les quatre premières minutes, le temps que les files se forment, le goulot théorique est affiché.

3.3 Protocole d'échange

Le backend expose un point d'accès WS `/ws`. Toutes les 100 ms, il envoie un message `state` au format JSON :

```
{ "type": "state", "speed": 4, "paused": false, "preset": "initial",
  "config":   { "times": [5, 8, 15, 7], "servers": [1, 1, 1, 1], ... },
  "stations": [{ "name": "Contrôle qualité", "state": "marche", "queue": 6,
                 "util": 0.96, "avail": 0.97, "servers": [{ "progress": 0.4 }] }],
  "products": [{ "id": 12, "st": 2, "s": "q", "k": 0 }],
  "kpi":      { "throughput_now": 3.9, "theoretical": 4.0, "produced": 120, ... },
  "events":   [{ "t": 31.0, "text": "Panne : Contrôle qualité (17 s)" }],
  "exits":    [[11, "ok"]] }
```

Listing 3.1. Message d'état (extrait).

Chaque produit est décrit par sa position logique (poste, `q` en file, `p` en traitement, `t` sur le convoyeur, et sa progression) : c'est la scène Unity qui la traduit en coordonnées 3D. Le backend reste ainsi indépendant de la représentation graphique. Dans l'autre sens, l'interface envoie des commandes courtes (`{"cmd": "set_time", "station": 2, "value": 8}`). L'historique, qui n'a pas besoin du temps réel, est servi par trois points d'accès REST : `/api/runs`, `/api/history` et `/api/export.csv`. Le protocole complet figure en annexe C.

3.4 Répartition des rôles

Table 3.2. Répartition des responsabilités et correspondance dans le code.

Membre	Responsabilité	Livrable dans le dépôt
Aymane Issami	3D & Architecture	Architecture globale, scène et implantation spatiale (`LineLayout`, `LineController`)
Marouan Ezbakhe	3D & Animation	Modèles des machines, animations, états visuels (`LineBuilder`, `StationUnit`)
Hiba Benzahra	Simulation industrielle	Logique de la ligne, cycles, pannes, données (`sim.py`)
Zaid Hsain	Backend & Communication	Serveur WS/REST, synchronisation, stockage (`server.py`, `storage.py`)
Houssam Kichchou	Dashboard & KPI	Supervision, KPI, historique et statistiques (`app.js`)
Mohamed Hamdouch	Optimisation & Validation	Scénarios, comparaison avant/après, tests (`compare_presets`, `ScenarioBenchmark`, `tests/`)

Conformément au point d'organisation du plan de travail, le protocole de la section précédente est le contrat entre les équipes 3D, simulation et backend : les états des machines et le format des messages y sont fixés, si bien que chaque brique peut évoluer sans casser les autres. Le découpage du code suit d'ailleurs exactement cette répartition : un fichier principal par responsabilité.

Chapitre 4

Implémentation

4.1 Organisation du dépôt

```
backend/   sim.py         modèle de la ligne et KPI
           server.py      FastAPI : WebSocket /ws, REST /api/*
           storage.py     historique SQLite
frontend/  index.html, style.css
           js/app.js      tableau de bord, historique, commandes
           unity/         build WebGL (généré, non versionné)
unity/     projet Unity 6 (URP)
           Assets/Scripts/Net       client WebSocket, DTO du protocole
           Assets/Scripts/Scene     ligne, machines, caméra, étiquettes
           Assets/Scripts/Bench     banc d'essai des scénarios
           Assets/Editor            construction de la scène, build WebGL
tests/     test_sim.py    cohérence du modèle (21 tests)
           test_api.py    WebSocket et REST (2 tests)
tools/     resultats_vers_markdown.py   résultats du banc d'essai
docs/      rapport/, presentation/, repartition/, cahier-des-charges.md,
           protocole-websocket.md, resultats-unity.md, manuel-
   utilisateur.md
```

Listing 4.1. Arborescence du dépôt.

L'application se lance par une seule commande (`python backend/server.py`) puis s'ouvre à l'adresse `http://localhost:8000`. Le backend sert aussi les fichiers de l'interface et le build Unity WebGL : aucun serveur web séparé n'est nécessaire. Les fichiers du build sont compressés (`.unityweb`, gzip) ; un petit intergiciel du backend ajoute l'en-tête `Content-Encoding` et le type MIME adapté pour que le navigateur les décompresse lui-même. Une variable d'environnement facultative, `SIM_SEED`, fixe la graine aléatoire pour rendre les pannes reproductibles.

4.2 Le modèle de simulation (`sim.py`)

La classe `Sim` contient l'état de la ligne : les files (`deque`), les machines de chaque poste (`Server` : produit en cours, temps restant, drapeau « terminé »), les produits en transit et les compteurs statistiques. La boucle principale `step(dt)` applique les règles décrites au chapitre 3 (figure 3.2). L'extrait suivant montre le cœur du traitement d'un poste : libération du produit terminé si la file aval a de la place, sinon blocage ; puis prise d'un nouveau produit et avancement du traitement.

```
limit = c["wip_limit"] if c["adaptive"] else c["buffer"]
for i in reversed(range(N)):
    for s in self.servers[i]:
        if s.done and not self._release(i, s, limit):
            self.blocked[i] += dt          # file aval pleine : poste
   bloqué
```

```
            continue
        if s.p is None and not self.down[i]:
            if i == 0:                      # alimentation : matière
illimitée
                s.p = Product(self.next_id, t); self.next_id += 1
            elif self.queues[i]:
                p = self.queues[i].popleft()
                self.wait_sum[i] += t - p.t_enq
                s.p = p
            else:
                continue                    # poste en attente
            s.rem = c["times"][i]
        if s.p is not None and not s.done and not self.down[i]:
            s.rem -= dt; self.busy[i] += dt
            if s.rem <= 1e-9:
                s.done = True
                if i == CONTROLE:           # 5 % de rejets au contrôle
                    s.p.reject = self.rng.random() < c["reject_rate"]
                self._release(i, s, limit)
```

Listing 4.2. Traitement des postes, de l'aval vers l'amont (`Sim.step`).

Le générateur aléatoire est propre à chaque simulation et initialisé par une graine : deux simulations de même graine et de mêmes paramètres produisent exactement les mêmes pannes et les mêmes rejets. C'est la condition d'une comparaison honnête entre scénarios (chapitre 5). La fonction `compare_presets` exécute chaque scénario pendant une heure simulée, sans affichage, en moins de 0,1 s par scénario.

4.3 Le backend (`server.py`, `storage.py`)

Le serveur FastAPI démarre une tâche asynchrone qui, toutes les 100 ms réelles, fait avancer la simulation du nombre de pas correspondant à la vitesse choisie (×4 par défaut, soit 8 pas de 0,05 s), puis diffuse l'état à tous les navigateurs connectés. Plusieurs postes peuvent superviser la même ligne simultanément. Les commandes reçues sont validées (`_station`, bornes des valeurs) ; une commande invalide renvoie un message `error`.

Le module `storage.py` enregistre l'historique dans trois tables SQLite :

— `runs` : une ligne par exécution (démarrage, scénario appliqué, remise à zéro), avec la configuration ;

— `samples` : un relevé des KPI et de l'état des postes par minute simulée ;

— `events` : pannes, remises en service et changements de réglages, transmis par un rappel (`on_event`) que la simulation appelle à chaque événement.

La comparaison de scénarios, plus coûteuse, est exécutée dans un fil séparé (`asyncio.to_thread`) pour ne pas interrompre la diffusion temps réel.

4.4 La scène 3D (Unity)

La scène est un projet **Unity 6** (pipeline URP, espace colorimétrique linéaire) construit uniquement à partir de primitives (boîtes, cylindres, sphères) et d'un maillage de trémie généré par code : aucun modèle externe n'est importé. Un outil d'édition (menu `Twin > Construire la scène`) crée la scène, les matériaux et les textures de façon reproductible,

ce qui évite de dépendre d'un fichier binaire modifié à la main. Chaque poste a une silhouette propre — trémie pour l'alimentation, broche rotative pour le traitement, caméra et faisceau de balayage pour le contrôle, presse et carton pour l'emballage — et porte un voyant aux couleurs des états de la figure 3.3, une barre de progression et une étiquette flottante (nom, état, temps de cycle, file). Quand un poste passe à plusieurs machines, la scène ajoute les machines en parallèle et un aiguillage du convoyeur.

Table 4.1. Scripts C# de la scène Unity.

Script	Rôle
`TwinClient`, `WsTransport`	Connexion WebSocket (natif dans l'éditeur, passerelle JavaScript en WebGL), décodage des messages, tampon d'états
`LineController`	Applique l'état reçu : machines, produits, palette, étiquettes de sortie
`LineBuilder`, `StationUnit`	Construction et animation des machines (broche, faisceau, presse, voyant, barre)
`OrbitCamera`	Caméra orbitale lissée, brouillard dépendant de la distance, retour à la vue globale
`WorldLabel`	Étiquettes TextMeshPro orientées vers la caméra
`LineBridge`	Pont page web ↔ Unity (sélection d'une machine, vue globale)
`ScenarioBenchmark`	Banc d'essai : joue chaque scénario et exporte les mesures

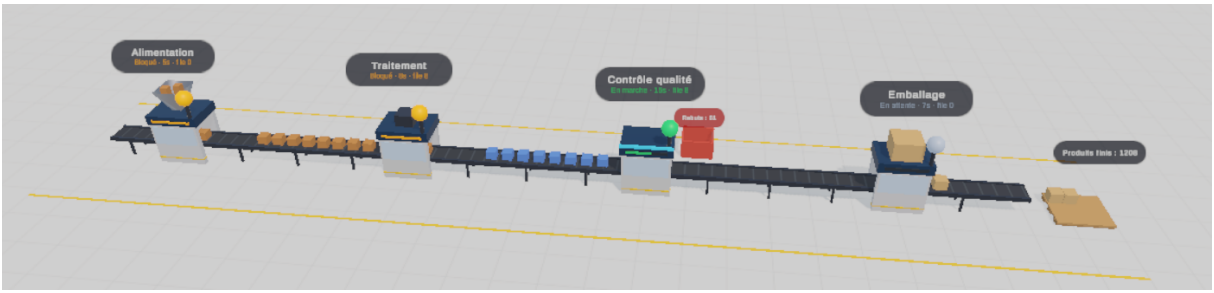


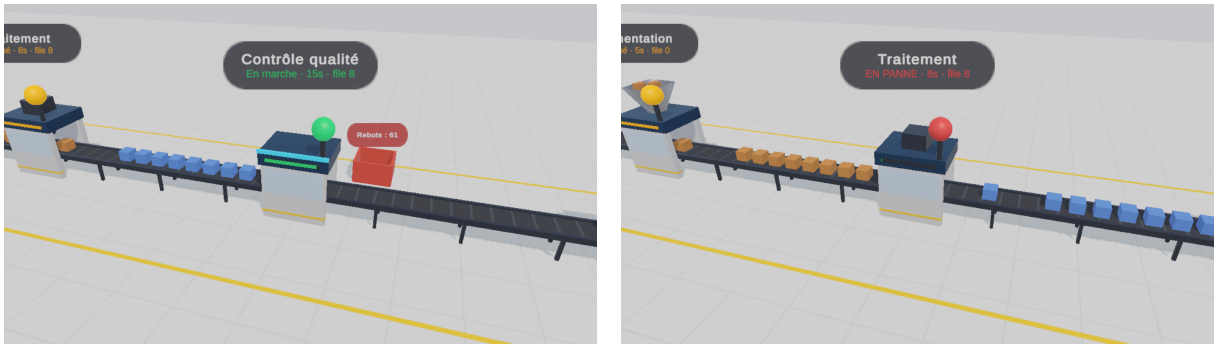
Figure 4.1. Vue d'ensemble de la scène Unity (build WebGL, ligne initiale). Les produits s'accumulent devant le contrôle qualité, le traitement et l'alimentation sont bloqués en amont ; le bac rouge reçoit les rebuts, la palette les produits finis.

4.4.1 Fluidité

Les produits ne sont pas déplacés par un calcul propre à la scène : le client conserve un tampon des derniers états reçus (10 Hz) et affiche la ligne **avec un léger retard de 0,15 s**, ce qui permet d'interpoler la position de chaque produit entre deux états consécutifs à la cadence de l'écran. Le mouvement reste ainsi continu même si un message arrive en retard. Les produits sont recyclés dans un *pool* (aucune allocation par image), les machines et le décor n'ont pas de collider inutile et le matériel du convoyeur est animé par un décalage de texture. Dans le build WebGL, le premier chargement prend quelques secondes (environ 12 Mo compressés).

4.4.2 Difficultés rencontrées

Deux points techniques méritent d'être signalés. D'abord, dans un build WebGL réduit au minimum, les colliders de capsule, sphère et maillage sont supprimés : créer une primitive par `GameObject.CreatePrimitive` à l'exécution produisait des erreurs dans la console du navigateur. Les formes créées pendant l'exécution partent donc de maillages partagés, sans collider ; seule la zone cliquable des quatre machines garde un `BoxCollider`. Ensuite, un brouillard trop dense voilait la scène dans une fenêtre étroite : il est désormais calculé à partir de la distance de la caméra.



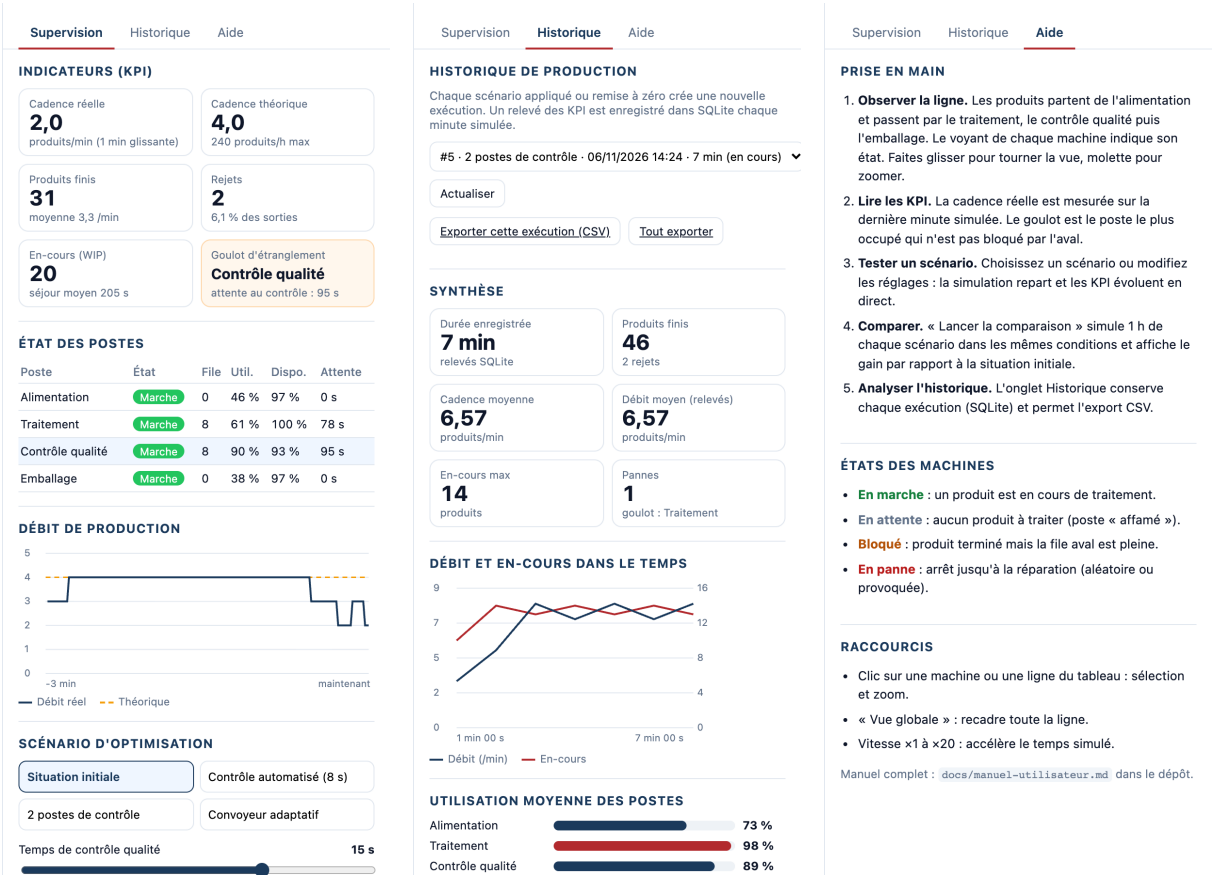
(a) Zoom sur une machine : étiquette d'état, voyant et barre de progression.

(b) Panne provoquée sur le traitement : voyant rouge, file amont pleine.

Figure 4.2. États visuels dans la scène Unity.

4.5 Le tableau de bord (`app.js`)

Le panneau latéral comporte trois onglets. **Supervision** affiche les KPI, le tableau des postes, la courbe de débit des trois dernières minutes avec le débit théorique, les réglages du scénario, la fiche de la machine sélectionnée (MTBF, MTTR, disponibilité) et le journal. **Historique** relit la base SQLite par l'API REST. **Aide** résume la prise en main.



(a) Supervision. **(b)** Historique (SQLite). **(c)** Aide.

Figure 4.3. Les trois onglets du tableau de bord.

4.6 Validation du modèle

Mohamed HAMDOUCH étant responsable de la validation, le modèle est vérifié par **23 tests automatisés** (pytest), exécutés en moins d'une minute. Ils ne vérifient pas que le code « tourne », mais que la simulation respecte des propriétés connues d'une ligne de production :

Table 4.2. Propriétés vérifiées par les tests automatisés.

Propriété vérifiée	**Résultat**
Débit de régime (sans pannes ni rejets) égal au débit théorique à 5 % près : 4,0 / 7,5 / 7,5 produits/min	3/3
Conservation : produits créés = finis + rejetés + en-cours, pour chaque scénario	4/4
Aucune file ne dépasse sa capacité (8, ou 3 en mode adaptatif)	4/4
Utilisation et disponibilité comprises entre 0 et 1	1/1
Les pannes réduisent la disponibilité et la production ; une panne provoquée arrête le poste	2/2
Taux de rejet observé proche de 5 % sur 4 h simulées	1/1
Le goulot est identifié au contrôle, puis déplacé au traitement après optimisation	1/1
Le convoyeur adaptatif réduit l'en-cours sans perte de débit	1/1
Comparaison reproductible à graine identique ; ajout et retrait de machines sans perte de produit	2/2
Historique : relevés, événements, export CSV ; API WS et REST	4/4
Total	**23/23**

Un contrôle supplémentaire est fourni par la **loi de Little** [12], $L = \lambda W$: l'en-cours moyen doit égaler le débit d'entrée multiplié par le temps de séjour moyen. Sur la ligne initiale avec pannes, $\lambda = (218 + 12)/3600$ produits/s et $W = 286$ s donnent $L \approx 18{,}3$, pour un en-cours moyen mesuré de 19,0. L'écart de 4 % s'explique par les produits encore en ligne en fin de simulation, dont le séjour n'est pas compté dans W.

Chapitre 5

Scénario d'optimisation

Objectif : réduire le goulot d'étranglement au poste de contrôle qualité afin d'augmenter le débit global de la ligne de production.

5.1 Problème identifié

Avec les paramètres initiaux (tableau 1.3), le contrôle qualité est le poste le plus lent : 15 s par produit contre 8 s au traitement. Le jumeau numérique rend le phénomène visible sans calcul (figure 4.1) : au bout de quelques minutes, la file devant le contrôle atteint sa capacité de 8 produits, le traitement se bloque faute de place, puis l'alimentation à son tour. Le tableau de bord confirme le diagnostic : le contrôle qualité est utilisé à 96 % du temps, contre 53 % pour le traitement et 35 % pour l'alimentation, et un produit y attend en moyenne près de deux minutes.

5.2 Solutions testées

1. **Contrôle automatisé** : le temps de contrôle passe de 15 à 8 s grâce à une inspection automatisée. Cadence théorique : 60/8 = 7,5 produits/min, le traitement (8 s) devenant le nouveau poste limitant.
2. **Deuxième poste de contrôle** : deux machines de 15 s en parallèle, soit un temps effectif de 7,5 s. Cadence théorique : 7,5 produits/min, limitée là aussi par le traitement.
3. **Convoyeur adaptatif** : le convoyeur ralentit devant une file chargée et la file maximale passe de 8 à 3 produits. Le débit théorique ne change pas (4 produits/min) : cette solution vise l'accumulation, pas la cadence.

5.3 Protocole de comparaison

Comparer deux scénarios en regardant l'animation n'est pas fiable : une panne tombée au mauvais moment dans l'un des deux suffit à inverser la conclusion. Chaque scénario est donc simulé **pendant une heure**, sans affichage, avec la **même graine aléatoire** et les mêmes paramètres de fiabilité, de transfert et de rejet. Deux séries de mesures existent. Le bouton « Lancer la comparaison » du tableau de bord exécute ce protocole hors ligne (fonction `compare_presets`, sortie conservée dans `docs/rapport/data/resultats.json`). Les chiffres de ce chapitre proviennent de la **seconde série, mesurée avec la scène Unity** : le banc d'essai (`ScenarioBenchmark`) joue chaque scénario pendant 3600 s simulées à la vitesse ×20, le backend étant lancé avec `SIM_SEED=7`, et lit les

indicateurs sur le flux WS réel qui anime la scène 3D (sortie dans `docs/rapport/data/resultats_unity.json`). Les valeurs sont donc celles que l'opérateur voit à l'écran.

5.4 Résultats

Table 5.1. Comparaison avant / après sur 1 h simulée, pannes activées (graine 7), mesurée avec la scène Unity.

Indicateur	Initiale	Contrôle 8 s	2 contrôles	Adaptatif
Produits finis / h	218	393 (+80 %)	**394** (+81 %)	218 (0 %)
Cadence moyenne (/min)	3,63	6,55	**6,57**	3,63
Cadence théorique (/min)	4,0	7,5	7,5	4,0
Attente au contrôle (s)	115	**9** (−92 %)	13 (−89 %)	40 (−65 %)
En-cours moyen (produits)	19,0	13,6 (−28 %)	14,7 (−23 %)	**9,4** (−51 %)
Temps de séjour moyen (s)	286	**115** (−60 %)	125 (−56 %)	145 (−49 %)
Rejets	12	22	23	12
Pannes / arrêts cumulés (s)	25 / 588	30 / 659	30 / 697	25 / 588
Goulot observé	Contrôle	Traitement	Traitement	Contrôle

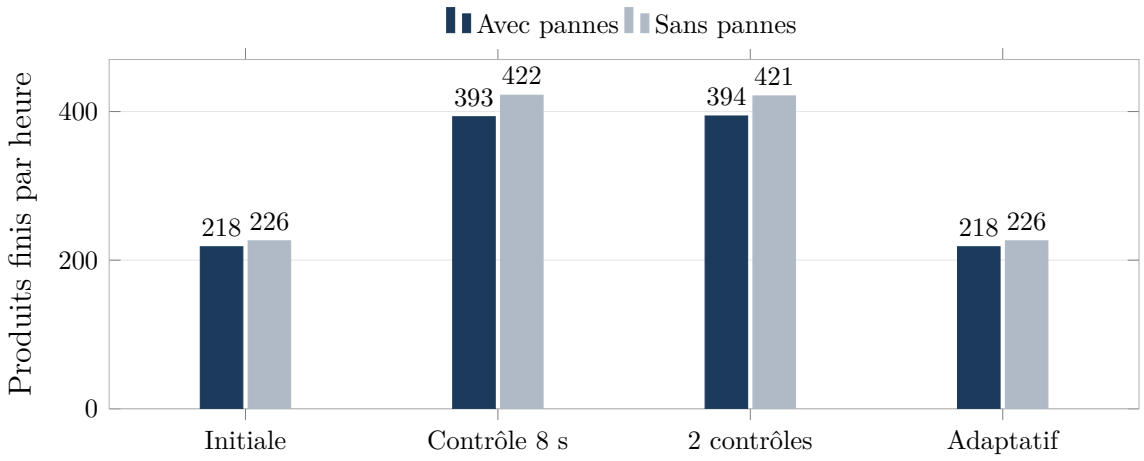


Figure 5.1. Production horaire par scénario, avec et sans pannes.

Indicateur	Situation initiale	Contrôle automatisé (8 s)	2 postes de contrôle	Convoyeur adaptatif	Réglages actuels
Produits / h	218	**390** **+79 %**	389 +78 %	218	389 +78 %
Cadence moy. (/min)	3,63	**6,50** **+79 %**	6,48 +79 %	3,63	6,48 +79 %
Attente contrôle (s)	115	**9** **-92 %**	16 -86 %	41 -65 %	16 -86 %
En-cours moyen	19,0	13,4 -30 %	14,9 -22 %	**9,4** **-51 %**	14,9 -22 %
Séjour moyen (s)	286	**113** **-60 %**	129 -55 %	145 -49 %	129 -55 %
Goulot	Contrôle qualité	Traitement	Traitement	Contrôle qualité	Traitement

Durée simulée : 1 h · pannes activées · % = écart vs situation initiale.

Figure 5.2. Le tableau de comparaison dans l'interface (bouton « Lancer la comparaison ») ; la colonne « Réglages actuels » reprend la configuration en cours.

5.5 Analyse

Le goulot se déplace. Les deux premières solutions augmentent la production d'environ 80 %, et le goulot passe du contrôle au traitement, comme prévu par le calcul théorique : le traitement est désormais utilisé à 94 % du temps (contre 53 % auparavant). Toute amélioration supplémentaire devra donc porter sur le traitement. C'est le principe central de la théorie des contraintes : optimiser un poste qui n'est pas le goulot ne sert à rien, et optimiser le goulot en fait apparaître un autre.

Contrôle automatisé ou deuxième poste ? À débit égal, le contrôle automatisé est légèrement meilleur sur l'attente (9 s contre 13 s) et sur le temps de séjour, parce qu'un produit traverse le contrôle en 8 s au lieu de 15. Sur une ligne réelle, le deuxième poste apporterait en plus une redondance : si l'une des deux machines tombe en panne, l'autre continue à 4 produits/min. Notre modèle ne capture pas cet avantage, car il traite la panne au niveau du poste entier (les deux machines s'arrêtent ensemble) ; c'est l'une des améliorations listées en perspective. Le choix entre les deux dépend donc d'informations que la simulation ne contient pas : coût de l'automatisation, coût d'un poste et d'un opérateur supplémentaires, criticité des arrêts.

Le convoyeur adaptatif ne produit pas plus. Il réduit l'en-cours de moitié et l'attente des deux tiers, sans perte de débit ; mais il ne déplace pas le goulot. Il a du sens en complément d'une des deux premières solutions (moins de stock entre postes, moins de produits immobilisés), pas à leur place.

Écart entre théorie et simulation. L'énoncé estimait 240 produits/h avant et 450 après. La mesure avec la scène Unity donne 218 et 393. Sans pannes, la ligne initiale produit 226 produits finis **et** 12 rejetés, soit 238 produits sortis du contrôle : l'écart restant avec 240 correspond au temps de démarrage (le premier produit n'atteint le contrôle qu'après 17 s, et le premier produit fini ne sort qu'après 41 s). Les pannes coûtent ensuite environ 3 % de production sur la ligne initiale et 7 % après optimisation : une ligne plus rapide perd davantage de produits pour une même minute d'arrêt. Enfin, l'objectif de 450 produits/h n'est pas atteint, même sans pannes (422 finis + 24 rejets = 446 sorties) : les

5 % de rejets et le démarrage suffisent à l'expliquer. Le jumeau numérique a donc corrigé l'estimation initiale à la baisse de 13 % (393 au lieu de 450), ce qui est précisément son rôle avant un investissement réel.

Cohérence entre la scène Unity et la simulation hors ligne. Les mesures faites avec la scène Unity ont été comparées à la sortie de `compare_presets` (tableau 5.2). La situation initiale et le convoyeur adaptatif donnent exactement la même production ; les séries sans pannes sont identiques ; les deux scénarios améliorés avec pannes s'écartent de 0,8 et 1,3 %. L'écart vient probablement du pas de simulation (0,05 s en ligne, 0,1 s hors ligne), qui décale légèrement les instants de panne. Il confirme que ce que l'opérateur voit à l'écran est bien le résultat du modèle validé, et non une seconde logique.

Table 5.2. Produits finis sur 1 h avec pannes : scène Unity et simulation hors ligne.

Scénario	Scène Unity	Hors ligne	Écart
Situation initiale	218	218	0,0 %
Contrôle automatisé (8 s)	393	390	0,8 %
2 postes de contrôle	394	389	1,3 %
Convoyeur adaptatif	218	218	0,0 %

Limite de la comparaison. Les mesures avec pannes reposent sur une seule graine aléatoire (7). Les tirages divergent dès que la séquence d'événements diffère (un produit de plus contrôlé consomme un tirage de rejet de plus), donc les pannes ne tombent pas exactement aux mêmes instants d'un scénario à l'autre. Pour des conclusions plus solides, il faudrait répéter la comparaison sur plusieurs graines et présenter des intervalles de confiance ; avec des écarts de l'ordre de 80 %, la conclusion principale ne fait cependant pas de doute.

5.6 Démonstration dans le jumeau numérique

1. **Avant optimisation** : lancer la situation initiale à vitesse $\times 10$; observer l'accumulation devant le contrôle, le blocage de l'amont et l'attente affichée (environ deux minutes).
2. **Incident** : provoquer une panne sur le traitement ; le voyant passe au rouge et la cadence chute.
3. **Après optimisation** : appliquer le contrôle automatisé à 8 s ou le deuxième poste ; la file se résorbe en quelques minutes et le goulot affiché passe au traitement.
4. **Comparaison** : lancer la comparaison, puis consulter et exporter les deux exécutions dans l'onglet Historique.

Chapitre 6

Aspects organisationnels et humains

Un jumeau numérique ne crée de la valeur que s'il est utilisé par les personnes qui pilotent la ligne. Ce chapitre répond à la troisième partie du cahier des charges, en se plaçant dans l'hypothèse d'un déploiement du jumeau dans un atelier réel.

6.1 Formation des opérateurs

Table 6.1. Parcours de formation proposé.

Public	Durée	Contenu	Support
Opérateurs de ligne	1 h	Lire la scène 3D (voyants, files, couleurs des produits), lire les KPI, signaler un écart entre le jumeau et la ligne	Onglet Aide, démonstration guidée
Chefs d'équipe	2 h	Piloter un scénario, provoquer une panne en formation, lire l'historique, exporter les données	Manuel utilisateur, exercices
Méthodes et maintenance	0,5 journée	Notion de goulot, comparaison de scénarios, interprétation des écarts théorie/simulation, MTBF et MTTR	Rapport, données CSV
Équipe informatique	0,5 journée	Architecture, protocole WS, base SQLite, ajout d'un poste, tests	README, code et tests

Le jumeau sert lui-même d'outil de formation : on peut provoquer une panne ou saturer un poste **sans aucun risque** pour la production, et montrer en quelques minutes simulées des phénomènes qui prendraient des heures à observer sur la ligne réelle.

6.2 Adaptation des processus de travail

— **Décision d'amélioration** : toute modification de la ligne (temps de cycle, ajout d'un poste, nouvelle règle de convoyage) est d'abord simulée et comparée dans les mêmes conditions que la situation actuelle ; la décision s'appuie sur le tableau de comparaison exporté.

— **Réunions de production** : la revue quotidienne s'appuie sur l'historique (débit par heure, pannes, goulot observé) plutôt que sur des relevés manuels.

— **Maintenance** : les MTBF et MTTR observés par poste orientent la maintenance préventive vers les postes dont la panne coûte le plus de production, c'est-à-dire en priorité le goulot.

— **Écart jumeau/réalité** : un processus de recalage est nécessaire. Lorsque la ligne réelle s'écarte durablement de la simulation, les paramètres du modèle (temps de cycle, MTBF, taux de rejet) sont mis à jour à partir des données mesurées.

6.3 Gestion du changement

L'introduction d'un outil de supervision peut être perçue comme un outil de surveillance individuelle. Quatre mesures limitent ce risque :

1. **Associer les opérateurs dès la conception** : ils connaissent les micro-arrêts et les contournements que les données ne montrent pas, et leur retour améliore la fidélité du modèle.
2. **Mesurer la ligne, pas les personnes** : les KPI affichés portent sur les postes et le flux, jamais sur un individu.
3. **Déployer progressivement** : d'abord en salle de formation, puis en supervision passive à côté de la ligne, avant de l'utiliser pour décider d'investissements.
4. **Montrer un premier gain concret** : le scénario du contrôle qualité, qui ne demande qu'une modification de poste pour un gain de près de 80 %, est un bon premier cas d'usage à présenter.

6.4 Organisation de l'équipe projet

Le projet a été mené par une équipe de six, organisée selon la répartition du tableau 3.2. Le travail suit les sept phases du plan de travail (annexe A) : analyse du besoin, modélisation 3D, simulation, backend, tableau de bord, optimisation et validation, rédaction. Le code est versionné dans un dépôt Git privé, et chaque responsabilité correspond à un fichier principal, ce qui limite les conflits entre membres. Les tests automatisés jouent le rôle de garde-fou commun : une modification qui casse une propriété du modèle est détectée avant d'être intégrée.

Chapitre 7

Conclusion et perspectives

7.1 Bilan des livrables

Table 7.1. Bilan des livrables du cahier des charges.

Livrable	Réalisation	Statut
Modèle de simulation du système physique	`sim.py` : 4 postes, files, blocages, pannes, rejets, KPI	Livré
Interface synchronisée avec des données simulées	Scène 3D Unity (WebGL) et tableau de bord sur un flux WS unique à 10 Hz	Livré
Scénario d'optimisation testé et documenté	3 solutions comparées sur 1 h simulée avec la scène Unity, chapitre 5	Livré
Historique et statistiques de production	SQLite, onglet Historique, export CSV	Livré
Validation du modèle	23 tests automatisés, contrôle par la loi de Little	Livré
Rapport, présentation, manuel utilisateur	Ce document, support de soutenance, `docs/manuel-utilisateur.md`	Livré

Le résultat principal est le suivant : sur une heure simulée avec pannes, réduire le temps de contrôle de 15 à 8 s, ou doubler le poste de contrôle, fait passer la production de 218 à environ 393 produits finis, soit +80 %, et déplace le goulot vers le traitement. Ces valeurs ont été mesurées avec la scène Unity connectée au backend et recoupent la simulation hors ligne à 1,3 % près. Le jumeau a aussi corrigé l'estimation théorique de départ (450 produits/h) en tenant compte des rejets, des transferts et des pannes.

7.2 Limites assumées

— **Les données sont entièrement simulées.** La ligne « physique » est elle-même un modèle ; les temps de cycle, MTBF et taux de rejet sont des hypothèses pédagogiques. Les conclusions valent pour ce modèle, pas pour une ligne réelle tant que ses paramètres n'ont pas été mesurés.

— **Pas de connexion à des équipements réels.** Il n'y a ni automate, ni capteur, ni protocole industriel : le jumeau n'est pas encore relié à un système physique au sens strict de Kritzinger et al. [5].

— **Pannes au niveau du poste.** Quand un poste a deux machines, une panne les arrête toutes les deux ; la redondance d'un poste doublé n'est donc pas modélisée.

— **Une seule graine par comparaison.** Les résultats ne sont pas accompagnés d'intervalles de confiance (chapitre 5).

— **Fluidité non chiffrée.** Le nombre d'images par seconde du build WebGL dépend du poste ; il doit être relevé sur la machine de démonstration.

— **Sécurité minimale.** L'application est prévue pour un usage local : pas d'authentification ni de chiffrement.

7.3 Perspectives

1. **Connecter une source réelle** : remplacer `sim.py` par un adaptateur OPC UA ou MQTT qui lit l'état d'automates, en conservant le même format de message ; la 3D et le tableau de bord n'auraient pas à changer.
2. **Recalage automatique** : estimer les temps de cycle et les MTBF à partir de l'historique réel et mettre à jour le modèle.
3. **Maintenance prédictive** : exploiter l'historique des pannes pour prévoir les arrêts et planifier la maintenance du goulot.
4. **Optimisation automatique** : explorer automatiquement l'espace des réglages (temps, nombre de machines, tailles de files) pour trouver le meilleur compromis débit/en-cours, sur plusieurs graines.
5. **Indicateurs industriels** : ajouter le Taux de Rendement Synthétique (TRS) (disponibilité $\times$ performance $\times$ qualité) par poste.
6. **Déploiement** : authentification, rôles (consultation ou pilotage), chiffrement, hébergement sur le réseau de l'atelier.

7.4 Compétences mobilisées

Ce projet nous a fait traverser toute la chaîne d'un jumeau numérique : modélisation d'un système de production et de ses phénomènes (goulot, blocage, fiabilité), programmation d'une simulation, communication temps réel entre un serveur et des clients, rendu 3D avec Unity exporté en WebGL, conception d'indicateurs et d'une interface de supervision, validation par des tests et par des lois connues, et travail en équipe autour d'un protocole commun. Il illustre ce qui distingue un jumeau numérique d'une simple animation : chaque chiffre affiché peut être relié au modèle, vérifié et reproduit.

Bibliographie et webographie

[1] Maria Zemzami. La révolution industrielle : le concept d'industrie 4.0 – projets dans le cadre de la thématique industrie 4.0 (cahier des charges). ENSAM Rabat, support de cours, 2026.

[2] Henning Kagermann, Wolfgang Wahlster, and Johannes Helbig. Recommendations for implementing the strategic initiative INDUSTRIE 4.0. Technical report, acatech – National Academy of Science and Engineering, Frankfurt am Main, 2013.

[3] Fei Tao, He Zhang, Ang Liu, and A. Y. C. Nee. Digital twin in industry : State-of-the-art. *IEEE Transactions on Industrial Informatics*, 15(4) :2405–2415, 2019.

[4] Michael Grieves. Digital twin : Manufacturing excellence through virtual factory replication. White paper, Florida Institute of Technology, 2014.

[5] Werner Kritzinger, Matthias Karner, Georg Traar, Jan Henjes, and Wilfried Sihn. Digital twin in manufacturing : A categorical literature review and classification. *IFAC-PapersOnLine*, 51(11) :1016–1022, 2018.

[6] ISO. ISO 23247-1 :2021 – automation systems and integration – digital twin framework for manufacturing – part 1 : Overview and general principles. International Organization for Standardization, 2021.

[7] Eliyahu M. Goldratt and Jeff Cox. *The Goal : A Process of Ongoing Improvement*. North River Press, 1984.

[8] Sebastián Ramírez. FastAPI. https://fastapi.tiangolo.com, 2018.

[9] Ian Fette and Alexey Melnikov. RFC 6455 : The websocket protocol. Internet Engineering Task Force, 2011. https://www.rfc-editor.org/rfc/rfc6455.

[10] Unity Technologies. Unity 6 – documentation, universal render pipeline et webgl. https://docs.unity3d.com, 2026.

[11] IEC. IEC 62541 – OPC unified architecture. International Electrotechnical Commission, 2020.

[12] John D. C. Little. A proof for the queuing formula : $L = \lambda W$. *Operations Research*, 9(3) :383–387, 1961.

Annexe A

Planning et ressources

A.1 Planning par phases

Table A.1. Phases du projet et responsables (planning prévisionnel en semaines).

Ph.	Travail	Responsable(s)	S1	S2	S3	S4	S5	S6	S7
1	Analyse du besoin, conception, protocole	Tous							
2	Modélisation et animation 3D	Issami, Ezbakhe							
3	Simulation industrielle	Benzahra							
4	Backend, communication, stockage	Hsain							
5	Dashboard et indicateurs	Kichchou							
6	Optimisation, tests, validation	Hamdouch							
7	Rapport et présentation	Tous							

A.2 Ressources

Table A.2. Ressources nécessaires.

Type	Ressource	Coût
Matériel	Ordinateurs personnels de l'équipe, navigateur récent compatible WebGL	0
Logiciels	Python 3.12, FastAPI, Uvicorn, pytest, SQLite, Unity 6 (licence personnelle gratuite)	0
Collaboration	Dépôt Git privé (GitHub), éditeurs de code	0
Documentation	LaTeX pour le rapport, HTML pour la présentation	0
Total		**0 MAD**

A.3 Critères d'évaluation

Table A.3. Critères du cahier des charges et éléments de preuve.

Critère	Élément de preuve
Atteinte des objectifs fixés	Les trois livrables sont réalisés (tableau 7.1)
Respect des délais et du budget	Planning par phases, aucun coût (tableau A.2)
Qualité de la réalisation technique	23 tests automatisés, loi de Little vérifiée, architecture découplée, résultats mesurés avec la scène Unity
Facilité d'utilisation pour les opérateurs	Scène 3D, voyants d'état, onglet Aide, manuel utilisateur

Annexe B

Résultats des tests automatisés

Sortie de `python -m pytest -rA` sur le dépôt (23 tests, tous réussis).

```
tests/test_api.py::test_websocket_etat_et_commandes                              PASSED
tests/test_api.py::test_api_historique_et_export                                 PASSED
tests/test_sim.py::test_debit_regime_egal_au_debit_theorique[initial-4.0]    PASSED
tests/test_sim.py::test_debit_regime_egal_au_debit_theorique[controle_auto-7.5]    PASSED
tests/test_sim.py::test_debit_regime_egal_au_debit_theorique[double_controle-7.5] PASSED
tests/test_sim.py::test_debit_theorique_calcule                                  PASSED
tests/test_sim.py::test_conservation_des_produits[initial]                       PASSED
tests/test_sim.py::test_conservation_des_produits[controle_auto]                 PASSED
tests/test_sim.py::test_conservation_des_produits[double_controle]               PASSED
tests/test_sim.py::test_conservation_des_produits[convoyeur_adaptatif]           PASSED
tests/test_sim.py::test_files_ne_depassent_pas_la_capacite[initial]              PASSED
tests/test_sim.py::test_files_ne_depassent_pas_la_capacite[controle_auto]    PASSED
tests/test_sim.py::test_files_ne_depassent_pas_la_capacite[double_controle] PASSED
tests/test_sim.py::test_files_ne_depassent_pas_la_capacite[convoyeur_adaptatif] PASSED
tests/test_sim.py::test_utilisation_et_disponibilite_bornees                     PASSED
tests/test_sim.py::test_pannes_reduisent_disponibilite_et_debit                  PASSED
tests/test_sim.py::test_panne_provoquee_arrete_le_poste                          PASSED
tests/test_sim.py::test_rejets_proches_du_taux_parametre                         PASSED
tests/test_sim.py::test_goulot_identifie_et_deplace                              PASSED
tests/test_sim.py::test_convoyeur_adaptatif_reduit_en_cours_sans_perte_de_debit PASSED
tests/test_sim.py::test_comparaison_reproductible                                PASSED
tests/test_sim.py::test_changement_de_machines_en_cours_de_route                 PASSED
tests/test_sim.py::test_stockage_historique_et_export_csv                        PASSED
======================== 23 passed in 2.11s ========================
```

Résultats sans pannes

Pour isoler l'effet des pannes, la comparaison a aussi été mesurée sans pannes avec la scène Unity (même graine) :

Table B.1. Comparaison sur 1 h simulée, pannes désactivées (scène Unity).

Indicateur	Initiale	Contrôle 8 s	2 contrôles	Adaptatif
Produits finis / h	226	422	421	226
Rejets	12	24	24	12
Cadence moyenne (/min)	3,77	7,03	7,02	3,77
Attente au contrôle (s)	114	0	0	40
En-cours moyen	19,2	12,2	13,0	9,5
Séjour moyen (s)	280	97	104	142
Utilisation du contrôle	100 %	100 %	93 %	100 %

Annexe C

Protocole d'échange

C.1 Commandes WebSocket (navigateur → backend)

Table C.1. Commandes acceptées sur `/ws`.

Action	Message
Appliquer un scénario	`{"cmd":"preset","name":"controle_auto"}`
Temps de cycle d'un poste	`{"cmd":"set_time","station":2,"value":8}` (1 à 60 s)
Machines en parallèle	`{"cmd":"set_servers","station":2,"value":2}` (1 à 3)
Convoyeur adaptatif	`{"cmd":"adaptive","value":true}`
Pannes aléatoires	`{"cmd":"failures","value":false}`
Provoquer / réparer	`{"cmd":"break","station":1}`, `{"cmd":"repair","station":1}`
Vitesse, pause, remise à zéro	`{"cmd":"speed","value":10}`, `{"cmd":"pause","value":true}`, `{"cmd":"reset"}`
Comparer les scénarios	`{"cmd":"compare","duration":3600}` → réponse `compare`

C.2 Messages backend → clients (page web et scène Unity)

— `hello` (à la connexion) : liste des postes et des scénarios disponibles.

— `state` (10 Hz) : `config`, `stations` (état, file, utilisation, disponibilité, attente, MTBF/MTTR observés, machines), `products` (poste, position `q`/`p`/`t`, progression), `kpi`, `history` (3 dernières minutes), `events`, `exits` (produits sortis ou rejetés depuis le message précédent).

— `compare` : `status` (`running` puis `done`) et résultats par scénario.

— `error` : commande invalide.

C.3 Points d'accès REST

Table C.2. API REST de l'historique.

Route	Réponse
`GET /api/state`	Instantané de l'état courant (JSON)
`GET /api/runs`	Liste des exécutions avec nombre de relevés et production
`GET /api/history?run=N`	Relevés par minute et événements de l'exécution `N`
`GET /api/export.csv[?run=N]`	Export CSV (séparateur `;`) d'une exécution ou de tout l'historique